République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Tunis

Ecole Nationale Supérieure d'Ingénieurs de Tunis

Code : GSP-RP-02-00
Création date : 18-05-2023

*Réf :*ING-1GInfo-01-2025

Computer Science Department

Summer Internship Report

First year in computer engineering

Publicly presented and defended

by

Yasmine Srioui

Training Management Website

Company

Company supervisor : Farhat Chammakhi

Internship period : from July 1 to July 31

Academic year : 2025-2026

5, Avenue Taha Hussein – Tunis
B. P. 56, Bab Menara 1008

Tel. : 71 . 496 . 066 :الهاتف
Fax : 71 . 391. 166: فاكس

5، شارع طه حسين ـ تونس
ص . ب . : 56 باب منارة 1008

Acknowledgments

I would like to express my sincere gratitude to the **École Nationale Supérieure des Ingénieurs de Tunis (ENSIT)** for the quality of its education and programs, and to our professors who dedicate their time to provide us with the best possible knowledge. Their support and guidance have been essential to the success of my first internship.

I am also very grateful to **ADS Valley** for welcoming me and offering me the opportunity to discover the professional environment and learn how projects are carried out in real time.

I express my heartfelt gratitude to my family for their continuous support in every aspect of my life.

Finally, I would like to thank the jury members for taking the time to evaluate my work.

Contents

List of Figures **v**

List of Tables **vi**

List of Acronyms **vii**

General introduction **1**

1 Project Overview **2**

Introduction 2

1.1 Context 2

1.2 Presentation of the Hosting Company 2

1.3 Objectives 3

Conclusion 4

2 System Design & Architecture **5**

Introduction 5

2.1 System Design 5

2.1.1 General Idea of the System 5

2.1.2 Use Case Diagram 5

2.2 System Architecture 6

2.2.1 Frontend Design 7

2.2.2 Backend Design 7

2.2.3 Database Design 7

Conclusion 8

3 Work environment& Implementation **9**

Introduction 9

3.1 Work Environment 9

3.1.1 Languages and Frameworks 9

3.1.2 Software Environment 10

3.2 Development Process 11

3.2.1 Frontend Implementation 12

3.2.2 Backend Implementation 14
3.2.3 Database Implementation 16
3.2.4 Integration 17
Conclusion 17

General Conclusion **18**

Bibliography **19**

List of Figures

2.1 Use Case Diagram of the Training Management Platform 6

3.1 React Logo 9
3.2 Nest Logo 10
3.3 Postgres Logo 10
3.4 VS Code Logo 10
3.5 Draw.io Logo 11
3.6 Postman Logo 11
3.7 Overleaf Logo 11
3.8 Admin Dashboard. 12
3.9 Available Courses Page 13
3.10 Appolo Client Setup 14
3.11 Resolver Code 15
3.12 Query Example in GraphQL Playground 15
3.13 Database connection configuration 16
3.14 Courses Table 17

List of Tables

1.1 Main services offered by ADS Valley[1] 3

List of Acronyms

- **API:** Application Programming Interface.
- **DB:** Database.
- **ERP:** Enterprise Resource Planning.
- **FP:** Functional Programming.
- **FRP:** Functional Reactive Programming.
- **IDE:** Integrated Development Environment.
- **JS:** JavaScript.
- **OOP:** Object-Oriented Programming.
- **PgAdmin:** PostgreSQL Administration Tool.
- **SQL:** Structured Query Language.
- **TS:** TypeScript.
- **UI:** User Interface.
- **UX:** User Experience.
- **VS Code:** Visual Studio Code.

General introduction

This summer, as a first-year Computer Engineering student, I had the opportunity to complete my very first internship, which lasted one month. This experience was extremely valuable, as it allowed me to gain insight into the professional environment of software engineers and to discover both the advantages and challenges of this career.

The internship marked an important transition from the theoretical foundations acquired at university to the practical application of modern and widely used development frameworks. It also represented my first exposure to full-stack web development using professional tools and workflows.

The main objective of the project was to design and implement a prototype of a **training management website**. The methodology adopted was primarily exploratory and iterative, relying on a self-learning approach combined with guidance from my supervisor.

This report is structured into four main chapters:

- **Chapter I: Project Overview**, Presents the hosting company, the context, and the objectives of the project.
- **Chapter II: System Design and Architecture**, Discusses the modeling and architectural design of the website from a beginner's perspective.
- **Chapter III: Work Environment and Implementation**, Describes the technologies, frameworks, and development process used.

Chapter 1:

Project Overview

Introduction

In this chapter, we present the hosting company, the context of the project, as well as the objectives of the internship.

1.1 Context

As a first-year Computer Engineering student, I had the opportunity to work on a small project during my summer internship with **ADS Valley**. The project consisted of designing a website to help training centers manage their courses, students, and trainers. These centers play a key role in helping individuals acquire new skills and adapt to the evolving job market. However, managing all processes manually can be complex and time-consuming, which highlights the need for a digital platform that automates administrative tasks and improves accessibility for learners.

It is within this context that my internship project was carried out. Despite my limited prior exposure to advanced frameworks, this challenge provided a valuable opportunity to explore modern web technologies while addressing a real and relevant problem.

The following section presents the hosting company.

1.2 Presentation of the Hosting Company

ADS Valley is a company specialized in web development with over ten years of experience in the e-commerce sector. It offers a complete range of services designed to ensure the success of digital projects. Its dynamic team, composed of experts in design, web development, marketing, and related fields, manages the entire website creation process from developing the brand identity to deploying the final product online. The company conducts an in-depth analysis of its clients' competitors to guarantee a leading position in their field of activity. A strong commitment to excellence lies at the heart of every stage of collaboration.[1]

Main Services Offered

Service	Description
Digital Strategy & Community Management	Personalized digital strategies tailored to each company's goals and target audience, including content creation and social media management.
Maintenance & Evolution	Regular technical maintenance and updates to ensure website performance, security, and long-term stability.
Branding and Graphic Design	Creation of unique visual identities including logos, graphic charters, and communication materials that reinforce brand recognition.
Enterprise Resource Planning (ERP)	Comprehensive optimization of business processes through customized ERP solutions, improving productivity and traceability.
Consulting and Expert Support	Strategic guidance at every stage of the digital project to maximize efficiency, innovation, and online impact.
Web Development	Design and development of high-performance, responsive websites centered on user experience and built with modern web technologies.

Table 1.1. Main services offered by ADS Valley[1]

1.3 Objectives

The primary objective of this internship project was to design and implement a prototype of a training management website capable of addressing the challenges faced by training centers. The platform aimed to offer a dual interface that meets the needs of both learners and administrators.

More specifically, the project sought to achieve the following goals:

- Develop a user interface that allows learners to browse available courses, enroll in programs, and track their progress intuitively.
- Build an administrator interface enabling training centers to add, edit, and manage courses, trainers, and student information efficiently.
- Gain practical experience with modern web development frameworks, particularly React.js for the frontend, NestJS for the backend, and GraphQL for data querying and management.

- Deliver a functional prototype that, while simple, could serve as a foundation for future improvements and extensions such as payment integration, certification features, or advanced reporting tools.

Conclusion

In summary, this first chapter outlined the context in which the project was developed and presented the objectives of the internship, both technical and educational. It highlighted the importance of this first professional experience in strengthening my skills and introducing me to modern development frameworks. The following chapter focuses on the system design and architecture, detailing the chosen structure, modeling, and technologies that served as the foundation for implementing the platform.

Chapter 2:

System Design & Architecture

Introduction

This chapter presents the system design and architecture of the training management platform. It includes the general design of the system, use case diagram, architecture layers, and descriptions of the frontend, backend, and database structure.

2.1 System Design

This section describes the overall system design and presents a use case diagram to illustrate the main interactions between users and the platform.

2.1.1 General Idea of the System

The platform aims to provide a functional system for managing courses. The specific goals are:

- Develop a user interface allowing students to browse available courses, enroll, and track their progress.
- Build an admin interface for adding, editing, and managing courses, trainers, and students.
- Deliver a functional system that, while basic, can serve as a foundation for future improvements.

2.1.2 Use Case Diagram

A UML use case diagram is a visual representation of the system's functionality from the end user perspective. It helps communicate the system behavior in terms of what users can do, without specifying how these actions are implemented.[2]

The key actors in the system include:

- **Admin**: Manages courses, trainers, and students.
- **Student**: Browses and enrolls into courses.

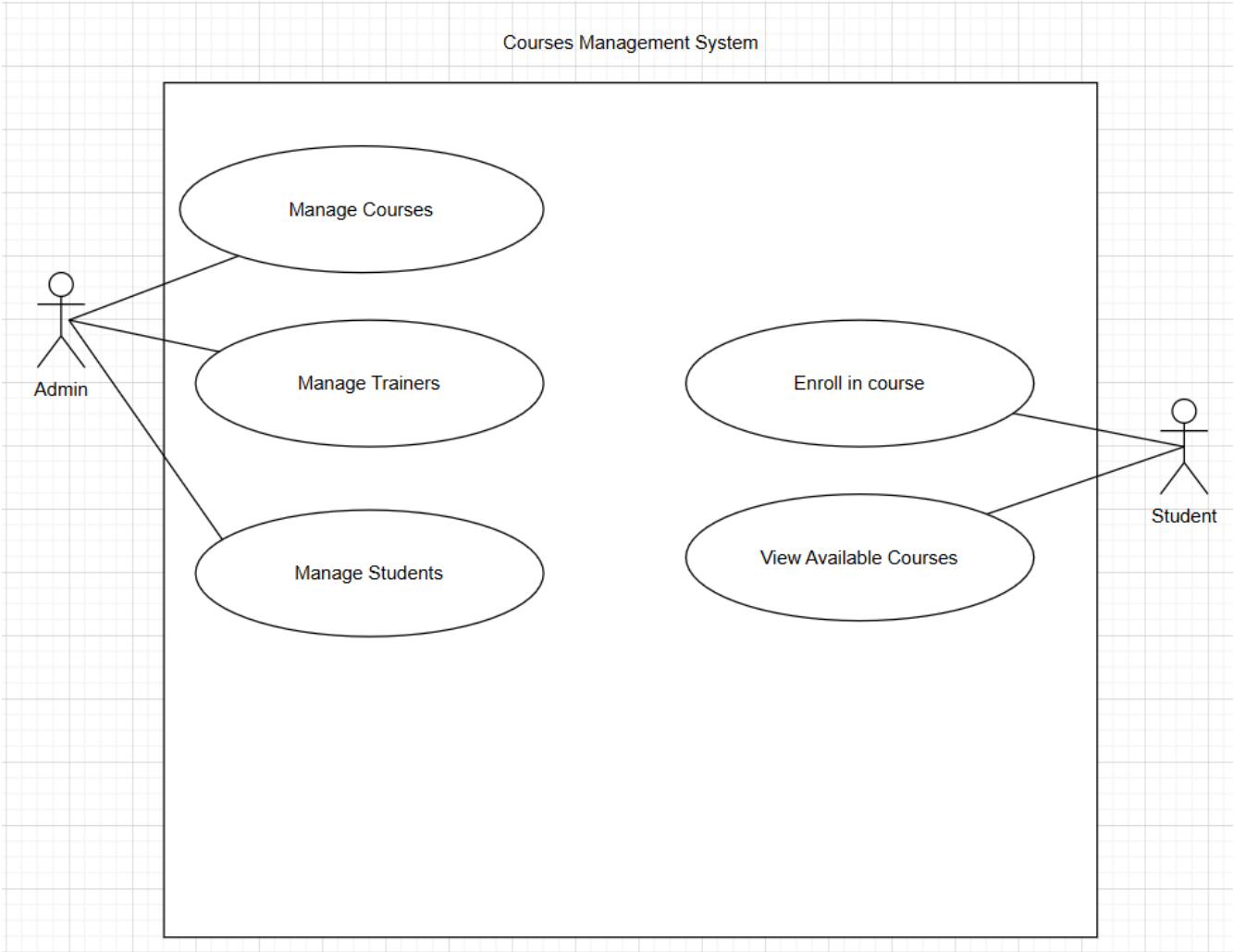

Figure 2.1. Use Case Diagram of the Training Management Platform

2.2 System Architecture

The platform follows a three-tier architecture that separates the user interface, application logic, and data storage. This design ensures better organization, maintainability, and scalability.

1. **Presentation Layer**
   - Responsible for displaying the user interface to students and administrators.
   - Enables students to browse and enroll in courses, and admins to manage courses, trainers, and students.
   - Communicates with the backend using GraphQL queries and mutations via Apollo Client.
2. **Application Layer**
   - Acts as an intermediary between the frontend and the database.

- Handles business logic, validates user actions, and executes database operations.
- Provides a GraphQL API for secure and structured data exchange.

3. **Data Layer**
   - Stores all persistent data, including courses, trainers and students.
   - Managed through TypeORM, which maps TypeScript entities to PostgreSQL tables and defines relationships.

2.2.1 Frontend Design

The frontend is implemented using React.js, focusing on reusability, responsiveness, and user experience.

Main components include:

- **Home Page**: Displays an overview of available courses.
- **Registration Form**: Allows students to enroll in courses.
- **Admin Dashboard**: Enables administrators to manage courses, trainers, and students.

Navigation is handled using React Router, and state management is implemented with React hooks (useState, useEffect).

2.2.2 Backend Design

The backend uses NestJS, which provides a modular and scalable architecture. Each functional area (courses, trainers, students) is implemented as a separate module to enhance maintainability.

Modules follow the NestJS layered architecture:

- **Controllers**: Define API endpoints and handle incoming requests, serving as an interface between the frontend and business logic.
- **Services**: Contain core business logic, process data, and interact with the database through TypeORM.
- **Entities**: Represent the data models that map to PostgreSQL tables, defining structure and relationships.

2.2.3 Database Design

The database is designed for data integrity, normalization, and efficient retrieval. Key entities include:

- **Students**: (id_student, name, email, etc.)

- **Trainers**: (id_trainer, name, expertise, email, etc.)
- **Courses**: (id_course, title, description, duration, price, etc.)

Relationships:

- A student can enroll in multiple courses.
- Each course is associated with one trainer.

TypeORM is used to manage these entities and relationships in NestJS, simplifying database operations through object-oriented models.

Conclusion

This chapter presented the design and architecture of the training management platform. It described the three-tier structure, frontend and backend components, and the database design. This phase allowed the development of a functional system while providing insight into the interactions between the various layers of a modern web application. The next chapter will focus on the user interfaces and system usage, illustrating how students and administrators interact with the platform.

Chapter 3:

Work environment& Implementation

Introduction

This chapter presents the work environment as well as the implementation process of the training management website developed during my internship. It details the development stage and the main features achieved.

3.1 Work Environment

This part introduces the languages and the frameworks used during the internship.

3.1.1 Languages and Frameworks

- **React.js:** React.js is a JavaScript library used for building user interfaces through reusable components.[3]

  Reason for choice: React offers fast rendering through its virtual DOM, strong community support, and easy integration with GraphQL via Apollo Client.

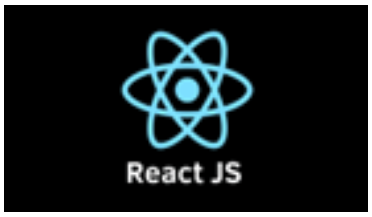

Figure 3.1. React Logo

- **NestJS:** NestJS is a framework for building efficient, scalable server-side applications using Node.js and TypeScript.[4]

  Reason for choice: NestJS provides strong structure, dependency injection, and TypeScript support.

Figure 3.2. Nest Logo

- **PostgreSQL:** PostgreSQL is an open-source object-relational database management system.[5]
  Reason for choice: PostgreSQL provides strong data integrity, scalability, and smooth integration with TypeORM used in NestJS.

Figure 3.3. Postgres Logo

3.1.2 Software Environment

- **Visual Studio Code:** A lightweight and extensible code editor used to write and debug both frontend and backend code.All React and NestJS code was written and tested within VS Code.[6]

  Reason for choice: It provides integrated terminal, and a large collection of extensions for JavaScript and TypeScript development.

Figure 3.4. VS Code Logo

- **Draw.io:** draw.io is a technology stack for building diagramming applications, and the world's most widely used browser-based end-user diagramming software. We have used to design the use case diagram.[7]

Reason for choice: It is free, simple, and integrates easily with Google Drive.

Figure 3.5. Draw.io Logo

- **Postman:** Postman is a collaborative API platform used to design, test, and document APIs. We have used to test backend APIs and verify GraphQL queries and mutations.[8]

  Reason for choice: It offers a user-friendly interface for sending requests, visualizing responses, and debugging API endpoints.

Figure 3.6. Postman Logo

- **Overleaf:**Overleaf is an online LaTeX editor that facilitates collaborative writing and formatting of academic reports.We have used for writing and formatting this internship report.[9]

  Reason for choice: It offers real time collaboration, version control, and professional formatting tools for academic writing.

Figure 3.7. Overleaf Logo

3.2 Development Process

In this section we will describe the details of the implementation process.

3.2.1 Frontend Implementation

The frontend was developed using React.js to ensure an interactive and dynamic user experience. It consists of two main interfaces:

- **Admin Interface:** Allows the management of courses, trainers, and students.As shown in Figure **??**, shows the dashboard-admin demonstrates the admin side and CRUD capabilities.

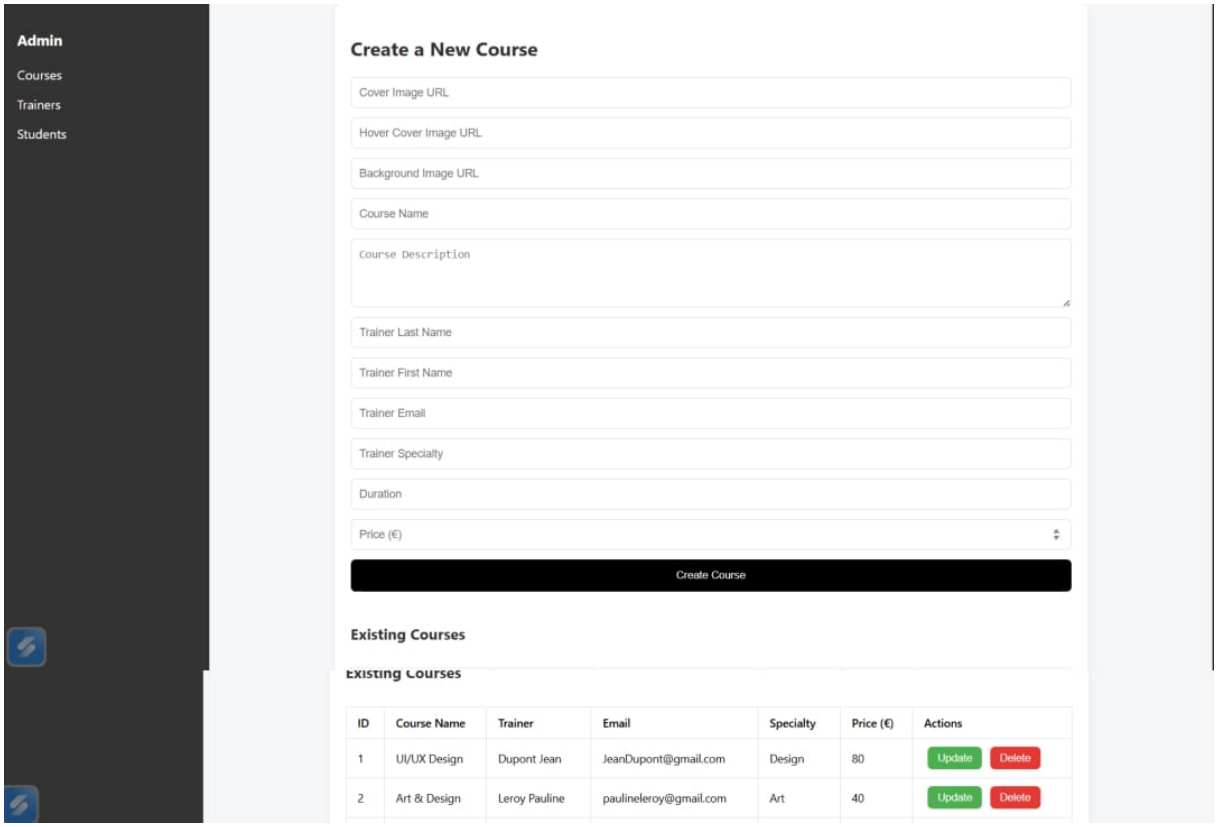

Figure 3.8. Admin Dashboard.

- **User Interface:** Allows learners to browse available courses, view details and enroll.As shown in Figure 3.9, the Available-Courses page lists all Courses retrieved through a GraphQL query.

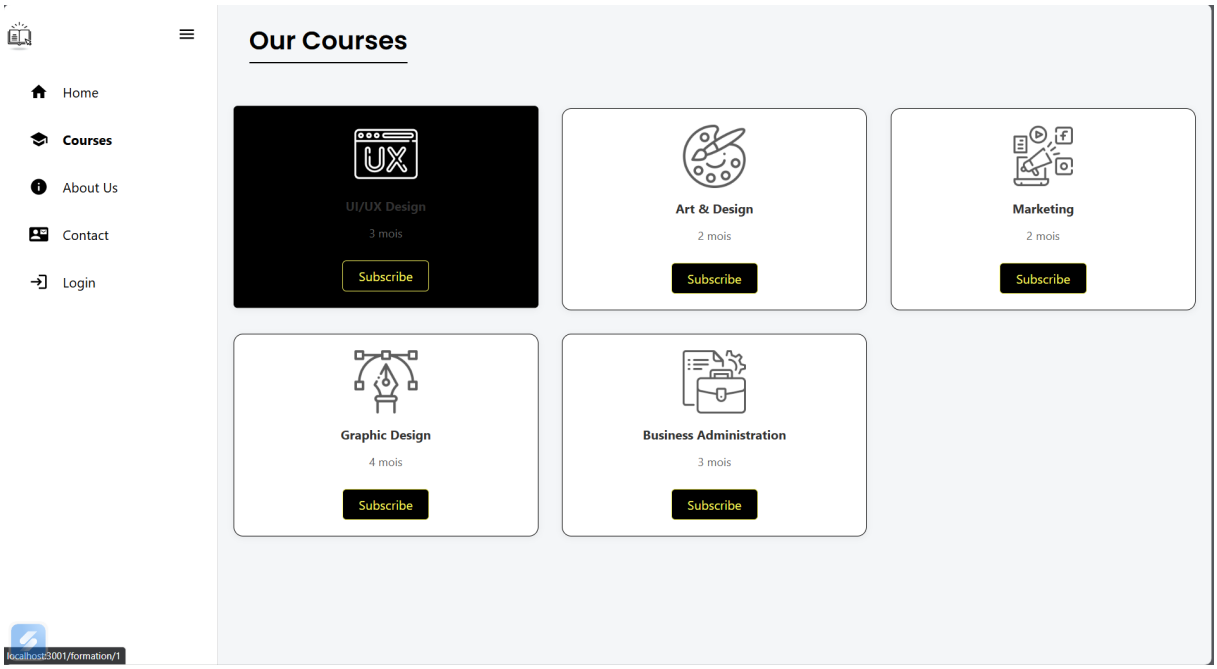

Figure 3.9. Available Courses Page

The interface is composed of reusable components such as course cards, registration forms, and dashboards. State management was used to handle user actions such as course enrollment. The frontend communicates with the backend through the Apollo Client, which manages GraphQL queries and mutations. As shown in Figure 3.10, a screenshot of Appolo Client setup.

```
import {
  ApolloClient,
  InMemoryCache,
  ApolloProvider,
} from "@apollo/client";
import { BrowserRouter } from 'react-router-dom';

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const client = new ApolloClient({
  uri: 'http://localhost:3000/graphql',
  cache: new InMemoryCache(),
});

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <ApolloProvider client={client}>
    <React.StrictMode>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </React.StrictMode>
  </ApolloProvider>
);
```

Figure 3.10. Appolo Client Setup

3.2.2 Backend Implementation

The backend was built with NestJS to manage business logic and connect the frontend to the database.

- Created GraphQL resolvers for Courses, Students and Trainers. As shown in Figure 3.11, code snippet from the resolver file related to the course entity.

```
import { Mutation, Args, Int } from '@nestjs/graphql';
import { FormationsService } from './formations.service';
import { Formation } from './formation.entity';
import { CreateFormationInput } from './dto/create-formation.input';
import { UpdateFormationInput } from './dto/update-formation.input';
import { Resolver, Query, ResolveField, Parent } from '@nestjs/graphql';
import { Client } from 'src/clients/client.entity';
import { ClientsService } from 'src/clients/clients.service';

@Resolver(() => Formation)
export class FormationResolver {
  constructor(private readonly formationsService: FormationsService,
    private clientsService: ClientsService,
  ) {}

 @Query(() => [Formation], { name: 'findAllFormations' })
  findAll(): Promise<Formation[]> {
    return this.formationsService.findAll();
  }

  @Query(() => Formation, { nullable: true })
  findOne(@Args('id', { type: () => Int }) id: number): Promise<Formation | null> {
    return this.formationsService.findOne(id);
  }

@Mutation(() => Formation)
createFormation(
  @Args('createFormationInput') createFormationInput: CreateFormationInput,
) {
  return this.formationsService.create(createFormationInput);
}
  @Mutation(() => Formation)
```

Figure 3.11. Resolver Code

- Implemented mutations to add, edit, and delete data.
- Implemented queries to fetch data for both user and admin interfaces. As shown in Figure 3.12, shows a query example in GraphQL Playground fetching courses data.

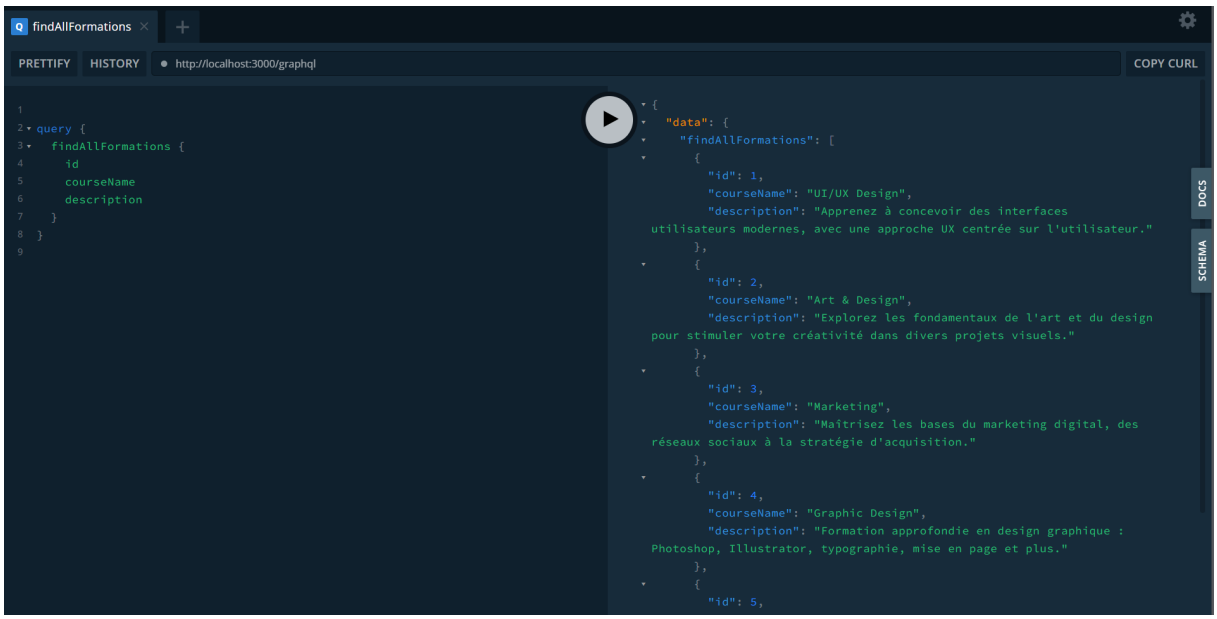

Figure 3.12. Query Example in GraphQL Playground

- Connected backend to PostgreSQL database using TypeORM. As shown in Figure **??**, configuration of the database connection using TypeORM.

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: 'localhost',
      port: 5432,
      username: 'postgres',
      password: 'postgres',
      database: 'formation_db',
      autoLoadEntities: true,
      synchronize: true,

    }),
    FormationsModule,
    ClientsModule,

    GraphQLModule.forRoot({
      driver: ApolloDriver,
  autoSchemaFile: join(process.cwd(), 'src/schema.gql'),
  playground: true,
}),
  ],
})
export class AppModule {}
```

Figure 3.13. Database connection configuration

3.2.3 Database Implementation

The database schema was designed to store the following entities: Students, Trainers and Courses.

- Primary and foreign keys were defined to maintain relationships.
- The structure supports all operations required by the application. As shown in Figure 3.14, data from courses table.

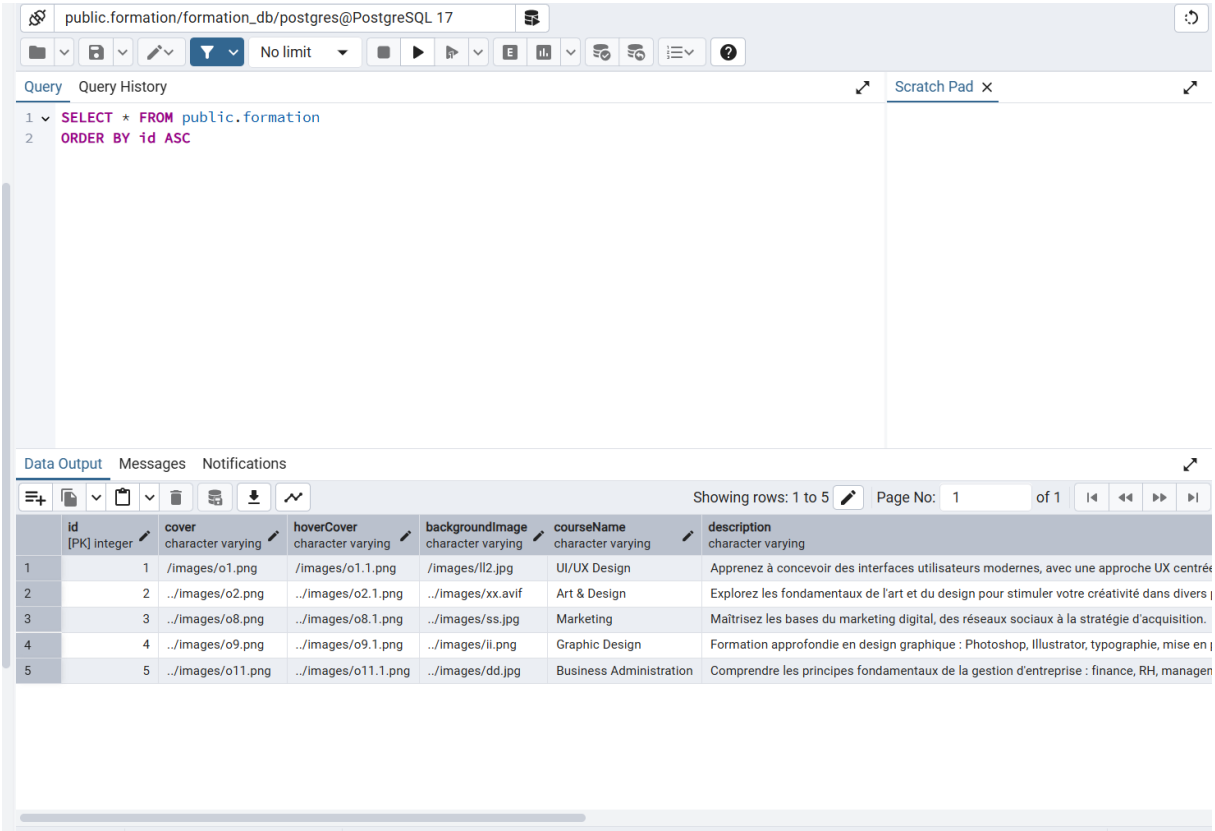

Figure 3.14. Courses Table

3.2.4 Integration

The frontend and backend were connected via GraphQL queries and mutations:

- Students can fetch and enroll in courses.
- Admins can manage courses, trainers, and Students.

Conclusion

This chapter presented the development environment, tools, and technologies used in the implementation of the training management platform. Despite being a prototype, the project effectively demonstrates core functionalities such as course management, student enrollment, and data synchronization between components.

General Conclusion

At the end of this internship, I had the opportunity to strengthen my technical skills and familiarize myself with tools commonly used in professional software development.

The main objective of the project was to create a functional system that allows administrators to manage courses, trainers, and students, and enables students to browse and register in a simple and intuitive way. By integrating **React.js** for the frontend, **NestJS** for the backend, and **PostgreSQL** for the database, the platform demonstrates a modular and scalable architecture suitable for future improvements.

This project also offered me the chance to gain practical experience with modern development tools such as **Postman**, and **PgAdmin** and to better understand the importance of communication between the different layers of a full-stack application. I also learned to overcome several technical challenges, including the configuration of GraphQL queries and mutations, data synchronization between the client and the server, and the management of component states in React.

Despite its success, the current prototype remains a simplified version of what could become a more complete learning management system. Future improvements could include adding authentication and authorization features, progress tracking dashboards,payment integration, certification features and automated notifications for learners and trainers.

In conclusion, this internship was a valuable professional and personal experience. It allowed me to develop not only my technical skills but also my problem-solving, time management, and teamwork abilities. It reinforced my motivation to continue improving as a software engineering student and to contribute to innovative digital projects in the future.

Bibliography

[1] "Ads valley – about us," https://adsvalley.io/a-propos/, accessed: 2025-10-15.

[2] V. Paradigm, "What is use case diagram?" 2023, accessed: 2025-10-15. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/

[3] "React – a javascript library for building user interfaces," https://reactjs.org/, accessed: 2025-10-15.

[4] "Nestjs documentation," https://docs.nestjs.com/, accessed: 2025-10-15.

[5] "Postgresql: The world's most advanced open source relational database," https://www.postgresql.org/, accessed: 2025-10-15.

[6] "Visual studio code," https://code.visualstudio.com/, accessed: 2025-10-15.

[7] draw.io, "About draw.io," https://www.drawio.com/about, accessed: 2025-10-15.

[8] "Postman api platform," https://www.postman.com/product/, accessed: 2025-10-15.

[9] Overleaf, "About overleaf," 2025, accessed: 2025-10-15. [Online]. Available: https://www.overleaf.com/about

Résumé

Ce projet vise à concevoir et développer un **site web de gestion des formations,** permettant aux administrateurs de gérer les formations et aux étudiants de s'inscrire aux formations. Le site a été réalisé en utilisant **React.js** pour le frontend et **NestJS** avec **PostgreSQL** pour le backend et la gestion de la base de données. L'application propose des fonctionnalités telles que la consultation des formations disponibles, l'inscription des étudiants et la gestion des formateurs.

Mots clés : Gestion des formations, React.js, NestJS, PostgreSQL, Inscription en ligne

Abstract

This project aims to design and develop a **web platform for courses management**, allowing administrators to manage courses and students to register into those courses. The website was implemented using **React.js** for the frontend and **NestJS** with **PostgreSQL** for backend and database management. The application provides features such as browsing available courses, student registration and trainers management.

Keywords: Training management, React.js, NestJS, PostgreSQL, Online registration

الملخص

يهدف هذا المشروع إلى تصميم وتطوير موقع ويب لإدارة الدورات التدريبية، يتيح للمسؤولين الإشراف الدورات وتنظيمها بكفاءة. تم تطوير الموقع باستخدام React.js في واجهة المستخدم، و NestJS مع PostgreSQL لإدارة الخادم وقاعدة البيانات. يوفر التطبيق مجموعة من الميزات، من بينها عرض الدورات المتاحة، تسجيل المتدربين، إدارة المدربين، ومتابعة تقدم المتدربين في الدورات.

الكلمات المفاتيح : إدارة الدورات، React.js، NestJS، PostgreSQL، التسجيل عبر الإنترنت، متابعة التقدم.